Airbnb Market Intelligence

Data Preprocessing Guide

Programme: Data Science AnalystLAB Internship
Project: Airbnb Market Intelligence Case Study
Tools: Python, Pandas, Databricks

---

1. Introduction

Data preprocessing is an important stage in the data science process. It involves inspecting, cleaning, transforming and preparing raw data before performing analysis or building machine learning models.

For this project, two Airbnb datasets were used:

1. **Listings Dataset:** Contains information about Airbnb properties, hosts, locations, prices and property characteristics.
2. **Reviews Dataset:** Contains information about listing reviews, review dates and reviewers.

The purpose of preprocessing was to improve data quality, ensure appropriate data types and prepare both datasets for exploratory data analysis, statistical testing and predictive modelling.

2. Importing the Required Libraries

The first step was importing the Python libraries required for data processing and analysis.

```
# Import Pandas for data manipulation
import pandas as pd

# Import NumPy for numerical operations
import numpy as np

# Import Matplotlib for data visualisation
import matplotlib.pyplot as plt
```

3. Loading the Datasets

The Listings and Reviews datasets were loaded into separate Pandas DataFrames.

```
# Load the Listings dataset
listing = pd.read_csv("listings.csv")

# Load the Reviews dataset
reviews = pd.read_csv("reviews.csv")
```

Each dataset was kept separate to make the cleaning and analysis process easier to manage.

4. Inspecting the Datasets

Before cleaning, the datasets were inspected to understand their structure.

```
# Display the first five Listings records
display(listing.head())

# Display the first five Reviews records
display(reviews.head())

# Check the dimensions of both datasets
print("Listings shape:", listing.shape)
print("Reviews shape:", reviews.shape)

# Inspect Listings data types
print(listing.dtypes)

# Inspect Reviews data types
print(reviews.dtypes)
```

This step helped identify:

- The number of rows and columns.
- Column names.
- Existing data types.
- Potential formatting inconsistencies.
- Fields requiring conversion.

5. Checking Missing Values

Missing values were investigated to identify incomplete records.

```
# Count missing values in Listings
print("Listings missing values:")
print(listing.isna().sum())

# Count missing values in Reviews
```

```
print("Reviews missing values:")
print(reviews.isna().sum())
```

Missing values were not automatically removed. Their impact was considered according to the purpose of each column.

For example, missing review ratings could affect rating analysis, while missing accommodation capacity could affect capacity-related analysis.

6. Cleaning the Listings Dataset

6.1 Cleaning Advertised Prices

The price column contained values that required conversion into a numerical format.

Currency symbols and thousands separators were removed before converting the column into numeric values.

```
# Convert prices to strings before cleaning
listing["price"] = listing["price"].astype(str)

# Remove dollar signs and commas
listing["price"] = listing["price"].str.replace(
    r"[$,]", "", regex=True
)

# Convert prices into numerical values
listing["price"] = pd.to_numeric(
    listing["price"], errors="coerce"
)
```

This allowed calculations such as averages, medians, correlations and predictive modelling.

Important: The currency of the advertised prices was not confirmed. Therefore, the analysis did not assume that prices from different countries represented the same currency.

6.2 Converting Numerical Columns

Some columns required conversion into numerical data types.

```
# List of numerical columns to clean
numeric_columns = [
    "bedrooms",
    "accommodates",
    "review_scores_rating"
```

```
]

# Convert each column into numeric format
for column in numeric_columns:

    # Invalid values become missing values
    listing[column] = pd.to_numeric(
        listing[column], errors="coerce"
    )
```

This ensured that numerical calculations could be performed correctly.

6.3 Handling Inconsistent Text Values

Some categorical fields contained the text value "null" rather than an actual missing value.

These values were identified and treated as missing information where appropriate.

```
# Replace the text "null" with actual missing values
listing["city"] = listing["city"].replace(
    "null", pd.NA
)

listing["room_type"] = listing["room_type"].replace(
    "null", pd.NA
)
```

This prevented invalid categories from being included in city and room type comparisons.

6.4 Inspecting Outliers

Advertised prices were examined using descriptive statistics.

```
# Display price summary statistics
print(listing["price"].describe())
```

The analysis identified unusually high advertised prices.

These values were not automatically deleted because they could represent genuine high-priced properties.

Medians were also used alongside averages to reduce the influence of extreme values when interpreting price distributions.

6.5 Checking Accommodation Capacity

The accommodates column was examined to understand the number of guests each listing could accommodate.

```
# Count listings by accommodation capacity
print(listing["accommodates"].value_counts(dropna=False))

# Examine the relationship between accommodation capacity and price
print(
    listing[["accommodates", "price"]].describe()
)
```

The bedrooms column was also examined. All 2,949 valid records contained one bedroom, meaning there was insufficient variation to investigate the effect of bedroom count on advertised prices.

7. Cleaning the Reviews Dataset

7.1 Converting Review Dates

The review date column was originally stored as text.

It was converted into a datetime format to support date-based analysis.

```
# Convert review dates into datetime format
reviews["date"] = pd.to_datetime(
    reviews["date"], errors="coerce"
)

# Inspect the converted dates
print(reviews["date"].dtype)

# Display the earliest and latest review dates
print("Earliest date:", reviews["date"].min())
print("Latest date:", reviews["date"].max())
```

The resulting date range was 3 June 2018 to 2 June 2019.

7.2 Checking Duplicate Records

Duplicate records and review identifiers were investigated.

```
# Check for completely duplicated records
print("Duplicate records:", reviews.duplicated().sum())

# Check for duplicate review IDs
```

```
print("Duplicate review IDs:", reviews["review_id"].duplicated().sum())

# Count unique review identifiers
print("Unique review IDs:", reviews["review_id"].nunique())
```

The dataset contained 10,000 unique review IDs and no duplicate review identifiers.

7.3 Checking Unique Listings

The number of unique listing identifiers was examined.

```
# Count unique listings represented in Reviews
print(
    "Unique listing IDs:",
    reviews["listing_id"].nunique()
)
```

The Reviews dataset contained 7,084 unique listing IDs.

8. Combining the Datasets

The Listings and Reviews datasets shared a common field called listing_id.

This identifier was used to connect review information to the corresponding listing records.

```
# Merge Listings and Reviews using listing_id
airbnb_combined = pd.merge(
    listing,
    reviews,
    on="listing_id",
    how="left"
)

# Display the combined dataset
display(airbnb_combined.head())

# Check the dimensions of the combined dataset
print(airbnb_combined.shape)
```

A left join was used to retain all records from the Listings dataset, including listings without matching review records.

Important finding: Only 42 listing IDs matched between the two datasets, and all matched listings were from Paris.

Therefore, the combined dataset contained many missing review fields. These missing values represented unmatched listing records and did not mean that the original Reviews dataset was empty.

9. Final Data Validation

After cleaning, the datasets were checked again to confirm that the transformations had been applied.

```
# Check Listings data types
print("Listings data types:")
print(listing.dtypes)

# Check Reviews data types
print("Reviews data types:")
print(reviews.dtypes)

# Check final dataset dimensions
print("Listings:", listing.shape)
print("Reviews:", reviews.shape)

# Check missing values after preprocessing
print("Listings missing values:")
print(listing.isna().sum())

print("Reviews missing values:")
print(reviews.isna().sum())
```

This validation helped confirm that the datasets were ready for further analysis.

10. Preprocessing Outcomes

The following results were obtained from the preprocessing stage:

Check	Result
Listings records	2,953
Listings columns	33
Reviews records	10,000
Reviews columns	4
Valid listing prices	2,949
Valid bedroom records	2,949
Valid accommodation records	2,949
Valid review ratings	2,284

Unique review IDs	10,000
Unique review listing IDs	7,084
Matching listing IDs	42

11. Conclusion

The preprocessing stage prepared the Airbnb Listings and Reviews datasets for further analysis.

The process involved inspecting the data, identifying missing values, converting data types, cleaning advertised prices, checking duplicates and investigating the relationship between the two datasets.

The cleaned datasets were subsequently used for exploratory data analysis, statistical hypothesis testing, predictive modelling and dashboard development.

An important outcome of preprocessing was identifying data quality limitations, particularly the low number of matching listing identifiers and the unconfirmed currency of advertised prices.

These limitations were considered during the interpretation of the project's findings.
